UNIT-2

C++ Tokens

In C++, tokens can be defined as the smallest building block of C++ programs that the compiler understands. It is the smallest unit of a program into which the compiler divides the code for further processing.

Types of Tokens in C++

We have 5 types of tokens each of which serves a specific purpose in the syntax and semantics of C++. Below are the main types of tokens in C++:

1. Identifiers

In C++, entities like variables, functions, classes, or structs must be given unique names within the program so that they can be uniquely identified.

It is recommended to choose valid and relevant names of identifiers to write readable and maintainable programs. Keywords cannot be used as an identifier because they are reserved words to do specific tasks. In the below example, "first_name' is an identifier.

string first_name = "Raju";

We have to follow a set of rules to define the name of identifiers as follows:

1. An identifier can only begin with a letter or an underscore (_).
2. An identifier can consist of letters (A-Z or a-z), digits (0-9), and underscores (_). White spaces and Special characters can not be used as the name of an identifier.
3. Keywords cannot be used as an identifier because they are reserved words to do specific tasks. For example, string, int, class, struct, etc.
4. Identifier must be unique in its namespace.
5. As C++ is a case-sensitive language so identifiers such as 'first_name' and 'First_name' are different entities.

2. Keywords

Keywords in C++ are the tokens that are the reserved words in programming languages that have their specific meaning and functionalities within a program. Keywords cannot be used as an identifier to name any variables.

For example, a variable or function cannot be named as 'int' because it is reserved for declaring integer data type.

There are 95 keywords reserved in C++.

3. Constants

Constants are the tokens in C++ that are used to define variables at the time of initialization and the assigned value cannot be changed after that. We can define the constants in C++ in two ways that are using the const, constexpr keyword and #define preprocessor directive.

const type name = val

constexpr type name = val

#define name val

4. Strings

In C++, a string is not a built-in data type like 'int', 'char', or 'float'. It is a class available in the STL library which provides the functionality to work with a sequence of characters, that represents a string of text.

string variable_name;
string name= "This is string";

When we define any variable using the 'string' keyword we are actually defining an object that represents a sequence of characters. We can perform various methods on the string provided by the string class such as length (), push_backk(), and pop_back().

5. Punctuators

Punctuators are the token characters having specific meanings within the syntax of the programming language. These symbols are used in a variety of functions, including ending the statements, defining control statements, separating items, and more.

Below are the most common punctuators used in C++ programming:

(i) Semicolon (;): It is used to terminate the statement.

(ii) (Square brackets []: They are used to store array elements.

(iii) Curly Braces {}: They are used to define blocks of code.

(iv) Double-quote ("): It is used to enclose string literals.

(v) Single-quote ('): It is used to enclose character literals.

Data Types

1. Basic Data Types

(i) int: Stores integers (whole numbers), typically 4 bytes.

(ii) char: Stores a single character, usually 1 byte.

(iii) float: Stores single-precision floating-point numbers (decimal numbers), typically 4 bytes.

(iv) double: Stores double-precision floating-point numbers, usually 8 bytes.

(v) bool: Stores Boolean values (true or false).

2. Derived Data Types

(i) Pointer types: Store memory addresses (e.g., int*, char*).

(ii) Array types: Collection of elements of the same type (e.g., int arr[10];).

(iii) Function types: Types for functions returning a value.

3. User-Defined Data Types

(i) struct: Group of variables under one name.

(ii) class: Like struct but with access control (private, public).

(iii) union: A special data type where all members share the same memory location.

(iv) enum: User-defined list of named integral constants.

4. Modifiers

4.1 You can modify basic types using:

(i) signed / unsigned: Whether the integer can be negative or not.

(ii) short / long: Size variations (e.g., short int, long int).

Example:

```
int a = 10;
char c = 'A';
float f = 3.14f;
double d = 3.14159265359;
bool flag = true;
```

Summary Table:

Type	Description	Typical Size
int	Integer numbers	4 bytes
char	Single character	1 byte
float	Single precision floating-point	4 bytes
double	Double precision floating-point	8 bytes
bool	Boolean (true/false)	1 byte
void	No value	0 bytes

Operators

C++ Operators are special symbols that are used to perform operations on operands such as variables, constants, or expressions. A wide range of operators is available in

C++ to perform a specific type of operations which includes arithmetic operations, comparison operations, logical operations, and more.

For example, `(A+B)`, in which 'A' and 'B' are operands, and '+' is an arithmetic operator which is used to add two operands.

There can be three types of operators:

1. Unary Operators

Unary operators are used with single operands only. They perform the operations on a single variable. For example, increment and decrement operators.

- Increment operator `(++)`: It is used to increment the value of an operand by 1.
- Decrement operator `(--)`: It is used to decrement the value of an operand by 1.

2. Binary Operators

They are used with the two operands and they perform the operations between the two variables. For example `(A<B)`, less than `(<)` operator compares A and B, returns true if A is less than B else returns false.

- Arithmetic Operators: These operators perform basic arithmetic operations on operands. They include `'+', '-', '*', '/',` and `'%'`
- Comparison Operators: These operators are used to compare two operands, and they include `'==', '!=', '<', '>', '<=', and '>='.`
- Logical Operators: These operators perform logical operations on boolean values. They include `'&&', '||', and '!'.`
- Assignment Operators: These operators are used to assign values to variables, and they include 'variables', `'-=', '*=', '/=', and '%='.`
- Bitwise Operators: These operators perform bitwise operations on integers. They include`'&', '|', '^', '~', '<<'and '>>'.`

3. Ternary Operator

The ternary operator is the only operator that takes three operands. It is also known as a conditional operator that is used for conditional expressions.

Expression1 ? Expression2 : Expression3;

If Expression1 became true then Expression2 will be executed
otherwise Expression3 will be executed.

Control Flow Statements.

Control flow refers to the order in which statements within a program execute. While programs typically follow a sequential flow from top to bottom, there are scenarios where we need more flexibility. This article provides a clear understanding about everything you need to know about Control Flow Statements.

What are Control Flow Statements in Programming?

Control flow statements are fundamental components of programming languages that allow developers to control the order in which instructions are executed in a program. They enable execution of a block of code multiple times, execute a block of code based on conditions, terminate or skip the execution of certain lines of code, etc.

Types of Control Flow statements in Programming:

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is true, and another block if the condition is false.
	switch-case	Evaluates a variable or expression and executes code based on matching cases.

Control Flow Statements Type	Control Flow Statement	Description
Jump Statements	break	Terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch.
	continue	Skips the current iteration of a loop and continues with the next iteration.

Conditional Statements in Programming:

Conditional statements in programming are used to execute certain blocks of code based on specified conditions. They are fundamental to decision-making in programs. Here are some common types of conditional statements:

1. If Statement in Programming:

The if statement is used to execute a block of code if a specified condition is true.

```
#include <stdio.h>
int main()
{
    int a = 5;
    if (a == 5) {
        printf ("a is equal to 5");
    }
    return 0;
}
Output
a is equal to 5
```

2. if-else Statement in Programming:

The if-else statement is used to execute one block of code if a specified condition is true, and another block of code if the condition is false.

```
#include <stdio.h>


int main()
{
    int a = 10;
    if (a == 5) {
        printf("a is equal to 5");
    }
    else {
        printf("a is not equal to 5");
    }
    return 0;
}
```

```
Output
a is not equal to 5
```

3. if-else-if Statement in Programming:

The if-else-if statement is used to execute one block of code if a specified condition is true, another block of code if another condition is true, and a default block of code if none of the conditions are true.

```
#include <stdio.h>


int main()
{
    int a = 15;
    if (a == 5) {
        printf("a is equal to 5");
```

```
    }
    else if (a == 10) {
        printf("a is equal to 10");
    }
    else {
        printf("a is not equal to 5 or 10");
    }
    return 0;
}
```

```
Output
a is not equal to 5 or 10
```

4. Ternary Operator or Conditional Operator in Programming:

In some programming languages, a ternary operator is used to assign a value to a variable based on a condition.

```
#include <stdio.h>

int main()
{
    int a = 10;
    printf("%s", (a == 5 ? "a is equal to 5"
                          : "a is not equal to 5"));
    return 0;
}
```

```
Output
a is not equal to 5
```

5. Switch Statement in Programming:

In languages like C, C++, and Java, a switch statement is used to execute one block of code from multiple options based on the value of an expression.

```
#include <stdio.h>
int main()
{
    int a = 15;
    switch (a) {
    case 5:
        printf("a is equal to 5");
        break;
    case 10:
        printf("a is equal to 10");
        break;
    default:
        printf("a is not equal to 5 or 10");
    }
    return 0;
}
```

Output

```
a is not equal to 5 or 10
```